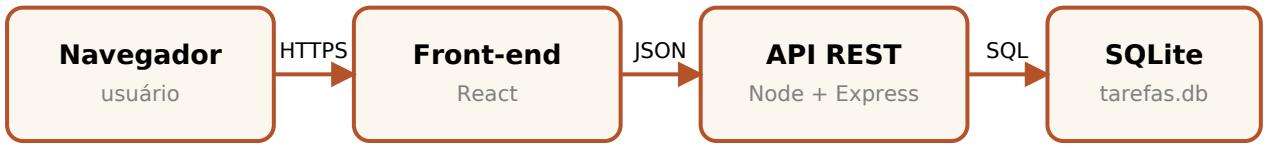
Aplicação de Tarefas
API REST com SQLite

Documentação técnica — Prova de Cloud · 2026

1. Objetivo

Desenvolver uma API básica com banco de dados SQLite, consumida por um front-end web de lista de tarefas (CRUD), hospedada de ponta a ponta e versionada no GitHub.

2. Arquitetura

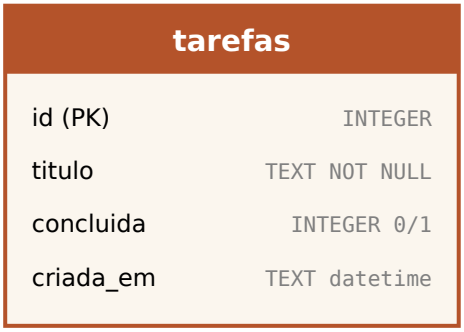


O usuário acessa o front-end React pelo navegador. O front-end faz requisições HTTP com JSON para a API Express, que lê e grava no arquivo SQLite armazenado em disco persistente.

3. Tecnologias

Camada	Tecnologia	Função
Front-end	React + TanStack Start + Tailwind	Interface da lista de tarefas
API	Node.js 22 + Express	Rotas REST e validação
Banco	SQLite (node:sqlite)	Persistência dos dados
Hospedagem	Lovable (front) + Render (API)	Publicação na nuvem
Versionamento	GitHub	Código-fonte e entrega

4. Modelo de dados



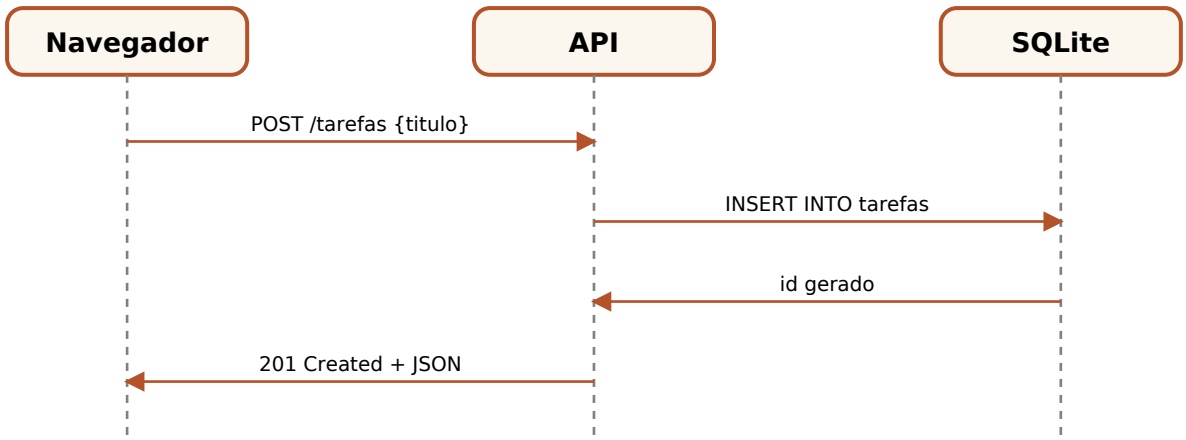
Script de criação:

```
CREATE TABLE IF NOT EXISTS tarefas (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  titulo TEXT NOT NULL,
  concluida INTEGER NOT NULL DEFAULT 0,
  criada_em TEXT NOT NULL DEFAULT (datetime('now'))
);
```

5. Endpoints da API

Método	Rota	Descrição	Status
GET	/	Status da API	200
GET	/tarefas	Lista todas as tarefas	200
GET	/tarefas/:id	Busca uma tarefa	200, 404
POST	/tarefas	Cria tarefa {titulo}	201, 400
PUT	/tarefas/:id	Atualiza titulo e/ou concluida	200, 400, 404
DELETE	/tarefas/:id	Remove tarefa	204, 404

6. Fluxo de criação de tarefa



7. Exemplos de uso

```
curl -X POST https://SUA-API.onrender.com/tarefas \
  -H "Content-Type: application/json" -d '{"titulo":"Estudar"}'
→ {"id":1,"titulo":"Estudar","concluida":false,"criada_em":"..."}
```

8. Hospedagem passo a passo

1. Conectar o projeto ao GitHub (Lovable → Configurações → Git); o código vai para um repositório.

2. No Render: New → Blueprint → selecionar o repositório. O arquivo render.yaml cria o serviço da API com disco persistente em /var/data para o SQLite.

3. Copiar a URL pública da API (ex.: https://api-tarefas.onrender.com) e testar GET /tarefas.

4. No front-end, definir VITE_API_URL com essa URL e publicar.

5. Compartilhar o link do repositório GitHub e a URL publicada.

9. Testes realizados

Foram testados localmente: criação (201), listagem, atualização de status (200), exclusão (204) e o fluxo completo pela interface, adicionando e concluindo tarefas.

10. Estrutura do repositório

```
api/server.js — API e criação do banco
api/package.json — dependências da API
render.yaml — configuração de hospedagem
src/routes/index.tsx — tela de tarefas
README.md — documentação com diagramas
```